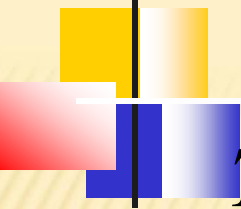


***Security at the
Transport Layer:
SSL and TLS***

CONTENTS

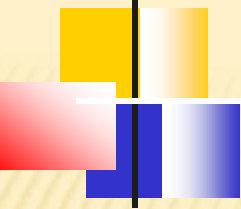
1. *Web Security Requirements*
2. *Secure Socket Layer*
3. *Transport Layer Security*
4. *Secure Electronic Transaction (SET)*
5. *Intruders,*
6. *Viruses*
7. *Related Threats*



The World Wide Web is fundamentally a client/server application running over the Internet and TCP/IP intranets

1. Web Security Considerations

- *The WEB is very visible.*
- *Web servers are easy to configure and manage.*
- *Complex software hide many security flaws.*
- *Users are not aware of the risks.*



Web Security Threats

Passive attacks: *eavesdropping* on network traffic between browser and server and gaining access to a Web site

Active attacks: *impersonating* another user, *altering* messages in transit between client and server, and *altering information* on a web site

Another way to classify Web security threats is in terms of the *location of the threat:*

*Web server,
Web browser, and
network traffic between browser and server*

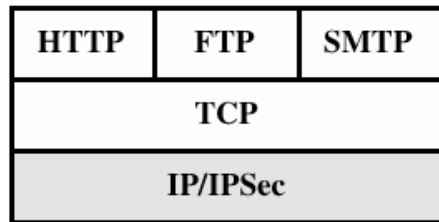
Issues of server and browser security fall into the category of computer system security

Issues of traffic security fall into the category of network security

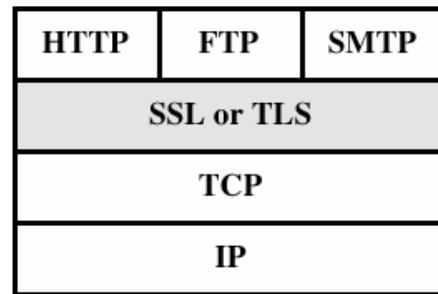
Table 7.1 A Comparison of Threats on the Web [RUBI97]

	Threats	Consequences	Countermeasures
Integrity	•Modification of user data •Trojan horse browser •Modification of memory •Modification of message traffic in transit	•Loss of information •Compromise of machine •Vulnerability to all other threats	Cryptographic checksums
Confidentiality	•Eavesdropping on the Net •Theft of info from server •Theft of data from client •Info about network configuration •Info about which client talks to server	•Loss of information •Loss of privacy	Encryption, web proxies
Denial of Service	•Killing of user threads •Flooding machine with bogus requests •Filling up disk or memory •Isolating machine by DNS attacks	•Disruptive •Annoying •Prevent user from getting work done	Difficult to prevent
Authentication	•Impersonation of legitimate users •Data forgery	•Misrepresentation of user •Belief that false information is valid	Cryptographic techniques

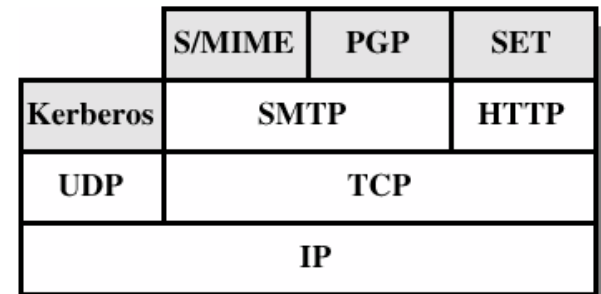
Web Traffic Security Approaches



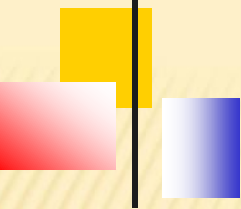
(a) Network Level



(b) Transport Level



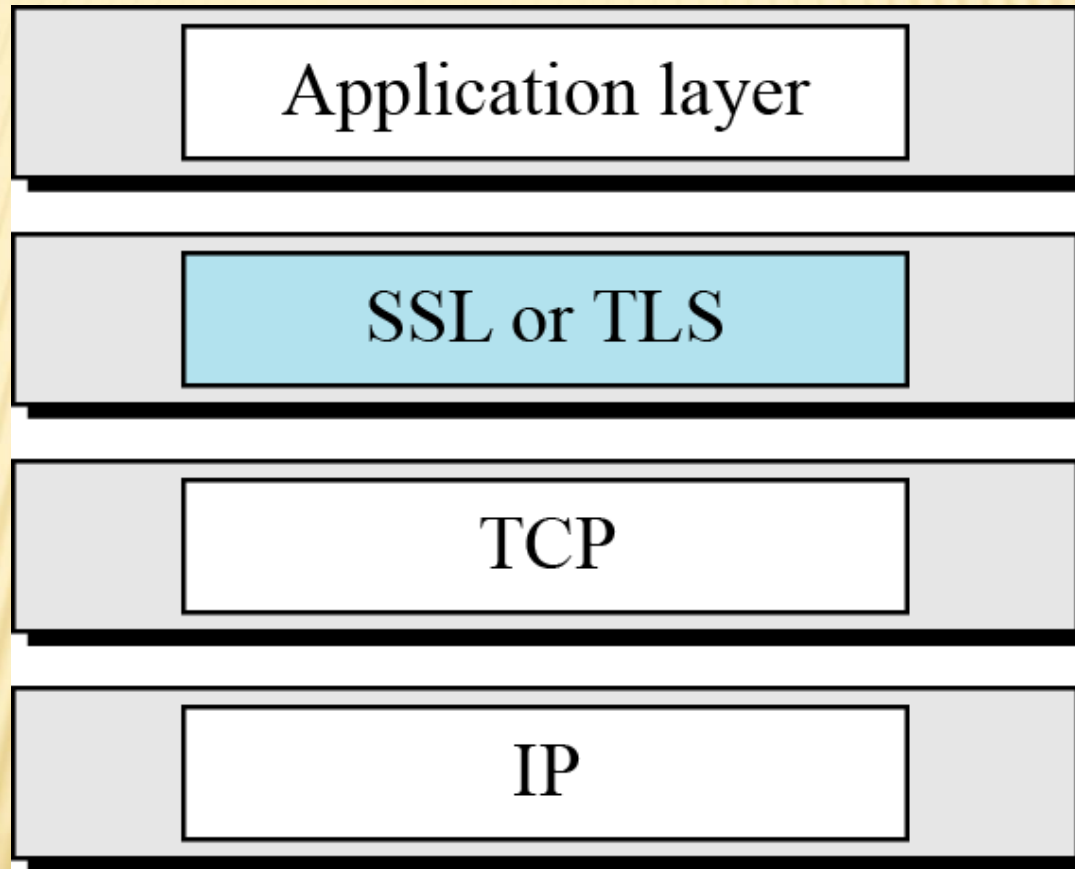
(c) Application Level



2. *SSL and TLS*

- **TLS (Transport Layer Security) and SSL (Secure Sockets Layer) – protocols**
- **provide data encryption and authentication between applications and servers, where that data is being sent across an insecure network**
- *SSL was originated by Netscape*
- *TLS working group was formed within IETF*
- *First version of TLS can be viewed as an SSLv3.1*

Figure *Location of SSL and TLS in the Internet model*



SSL ARCHITECTURE

*SSL is designed to provide **security and compression** services to data generated from the application layer.*

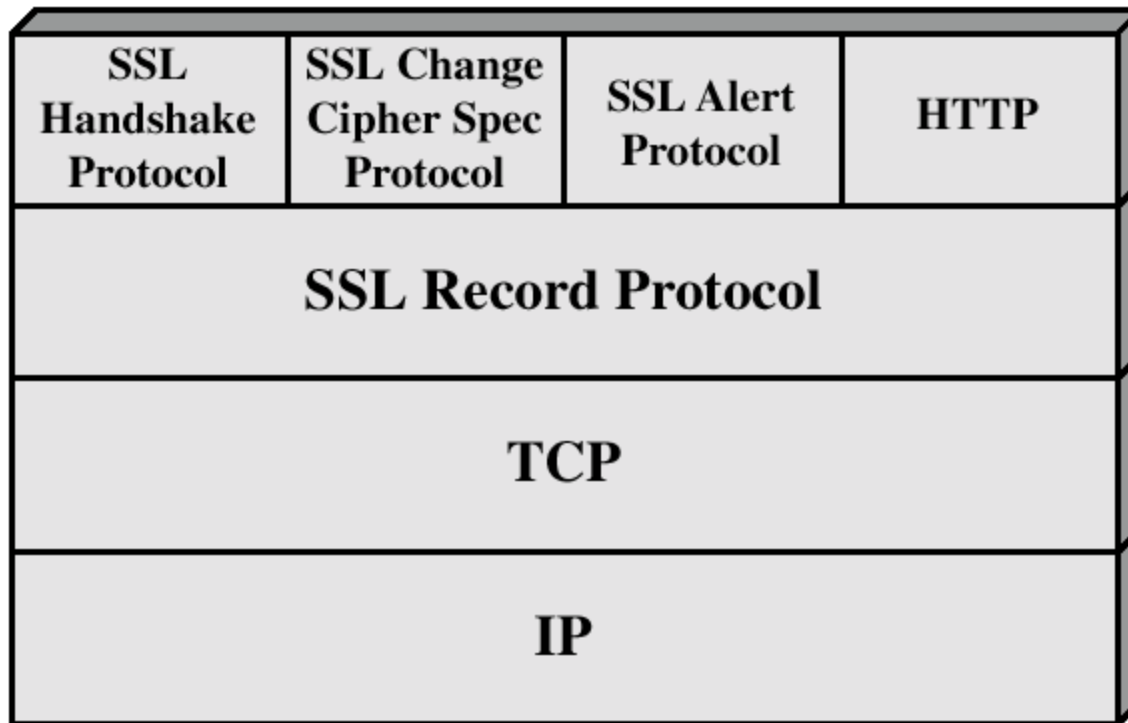


Figure 7.2 SSL Protocol Stack

Services

- ***Fragmentation***
- ***Compression***
- ***Message Integrity***
- ***Confidentiality***
- ***Framing***

Sessions and Connections

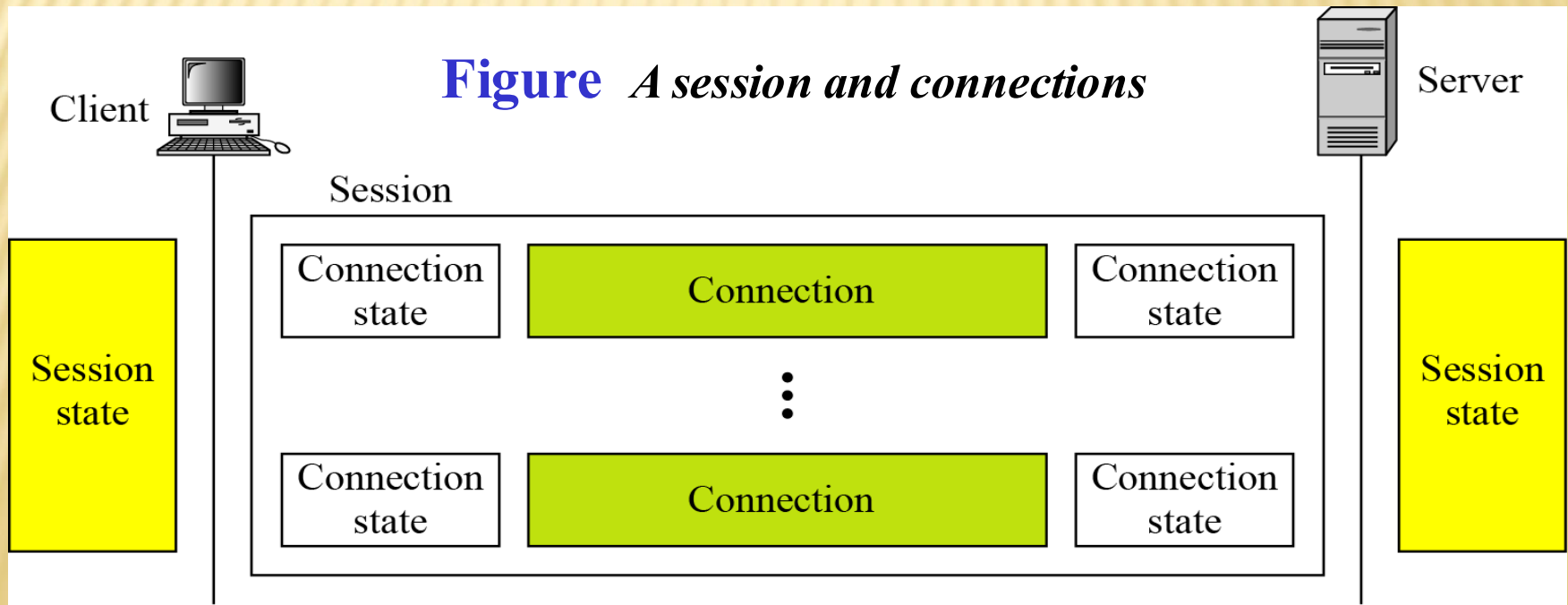
2 important SSL concepts - SSL Connection and SSL Session.

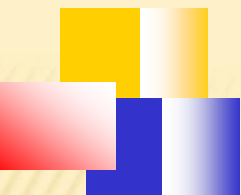
Connection:

- *It is a transport that provides a suitable type of service*
- *For SSL, connections are peer-to-peer relationships, transient*
- *Every connection is associated with one session*

Session:

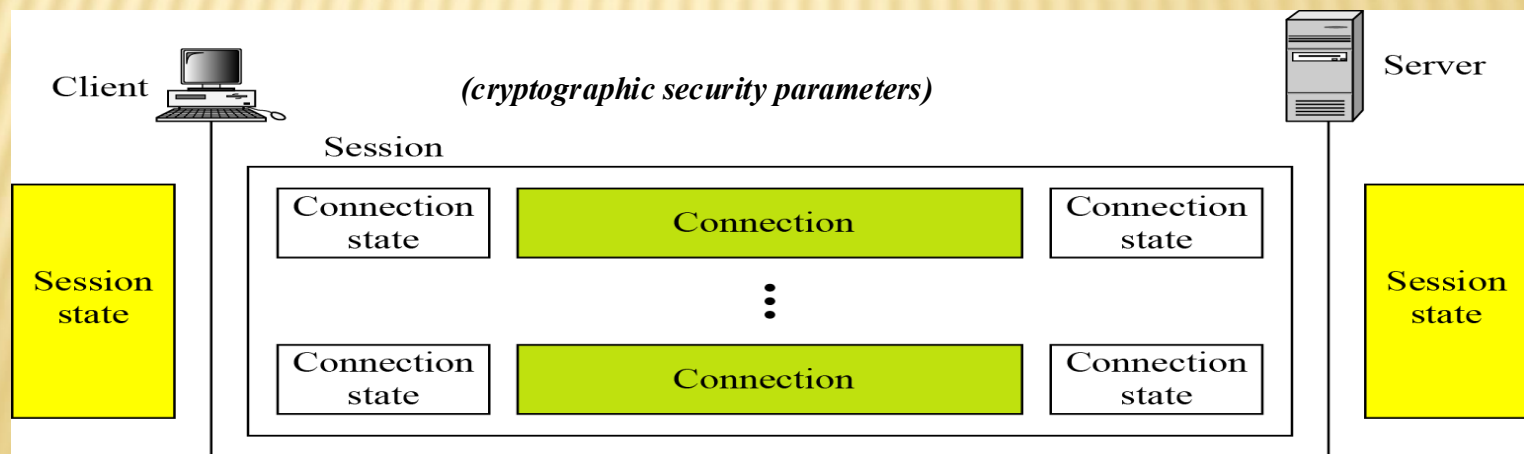
- *It is an association between a client and a server*
- *Sessions are created by the **Handshake protocol***





Connection → live communication channel,
Session → set of negotiated cryptography parameters

- ✓ Sessions define a set of **cryptographic security parameters**, which can be shared among multiple connections
- ✓ There are actually a number of states associated with each session
- ✓ During the Handshake protocol, **pending read and write states** are created
- ✓ Once a session is established, there is a **current operating state** for both read and write (i.e., receive and send)
- ✓ At the end of the Handshake protocol pending states become the current states



Session state parameters

✓ *A session state is defined by the following parameters*

Table *Session state parameters*

<i>Parameter</i>	<i>Description</i>
Session ID	A server-chosen 8-bit number defining a session.
Peer Certificate	A certificate of type X509.v3. This parameter may be empty (null).
Compression Method	The compression method.
Cipher Suite	The agreed-upon cipher suite.
Master Secret	The 48-byte secret.
Is resumable	A yes-no flag that allows new connections in an old session.

CONNECTION STATE PARAMETERS

Server and client random	Byte sequences that are chosen by the server and client for each connection
Server write MAC secret	The secret key used in MAC operations on data sent by the server.
Client write MAC secret	The secret key used in MAC operations on data sent by the client
Server write key	The conventional encryption key for data encrypted by the server and decrypted by the client.
Client write key	The conventional encryption key for data encrypted by the client and decrypted by the server
Initialization vectors	When a block cipher in CBC mode is used, an initialization vector (IV) is maintained for each key. This field is first initialized by the SSL Handshake Protocol. Thereafter the final ciphertext block from each record is preserved for use as the IV for the next record.
Sequence numbers	Each party maintains separate sequence numbers for transmitted and received messages for each connection. When a party sends or receives a change CipherSpec message, the appropriate sequence number is set to zero.

SSL Record Protocol- provide basic security services to various higher layer protocols

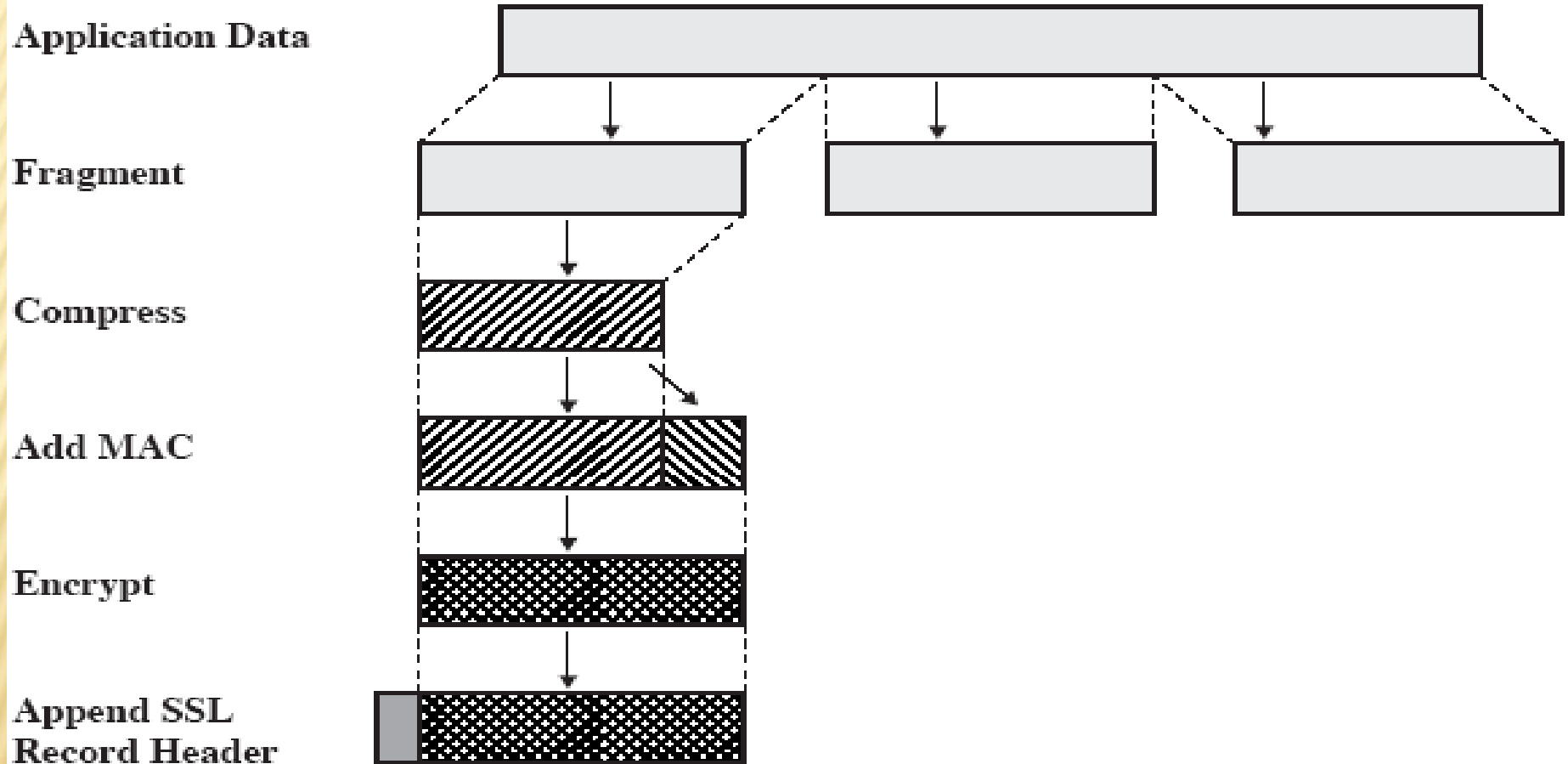


Figure 17.3 SSL Record Protocol Operation

SSL Record Protocol : 2 services

The SSL Record Protocol provides two services for SSL connections

1. **Confidentiality:** *The handshake protocol **defines a shared secret key** that is used for conventional encryption of SSL payloads*
 2. **Message Integrity:** *The handshake protocol also defines a shared secret key that is used to form a MAC*
- ✓ *The Record protocol takes an application message to be transmitted.*
- *fragments the data into manageable blocks,*
 - *optionally compress the data,*
 - *applies a MAC,*
 - *encrypts,*
 - *adds a header, and*
 - *transmits the resulting unit in a TCP segment.*
- ✓ *Received data is decrypted, verified, decompressed, and reassembled and then delivered to higher level users.*

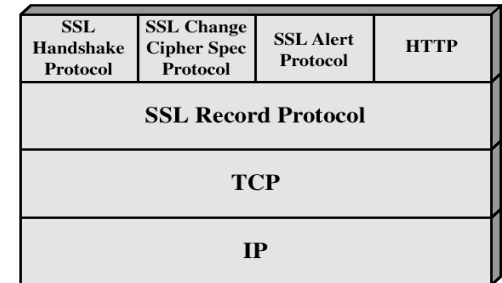


Figure 7.2 SSL Protocol Stack

Step 1: Fragmentation. Each upper layer message is fragmented into blocks of 2^{14} bytes (16,384 bytes) or less

Step 2: Compression is optionally applied. Compression must be lossless and may not increase the content length by more than 1024 bytes

Step 3: compute a MAC over the compressed data. For this, a shared secret key is used.

*hash(MAC_write_secret || pad_2 ||
hash(MAC_write_secret || pad_1 || seq_num ||
SSLCompressed.type || SSLCompressed.length || SSLCompressed.fragment))*

where

|| = concatenation

MAC_write_secret = shared secret key

hash = cryptographic hash algorithm; either SHA-1 or MD5

pad_1 = the byte 0x36 (0011 0110) repeated 48 times (384 bits) for MD5 and 40 times (320 bits) for SHA-1

pad-2 = the byte 0x5C (0101 1100) repeated 48 times for MD5 and 40 times for SHA-1

seq_num = the sequence number for this message

Go to HMAC

SSLCompressed.type = the higher level protocol used to process this fragment

SSLCompressed.length = the length of the compressed fragment

SSLCompressed.fragment = the compressed fragment (if compression is not used, the plaintext fragment)

Step 4: (Compressed message + MAC) are encrypted using symmetric encryption. Encryption may not increase the content length by more than 1024 bytes, so that the total length may not exceed $2^{14} + 2048$.

Block Cipher		Stream Cipher	
Algorithm	Key size	Algorithm	Key size
IDEA	128		
RC2-40	40		
DES-40	40	RC4-40	40
DES	56	RC4-128	128
3DES	168		
Fortezza	80		

Step 5: The final step of SSL Record Protocol processing is to prepend a header, consisting of the following fields:

- **Content Type(8 bits):** The higher layer protocol used to process the enclosed fragment
- **Major Version(8 bits):** Indicates major version of SSL in use. For SSLv3, the value is 3
- **Minor Version(8 bits):** Indicates minor version in use. For SSLv3, the value is zero
- **Compressed Length(16 bits):** The length in bytes of the plaintext fragment (or compressed fragment if compression is used). The maximum value is $2^{14} + 2048$

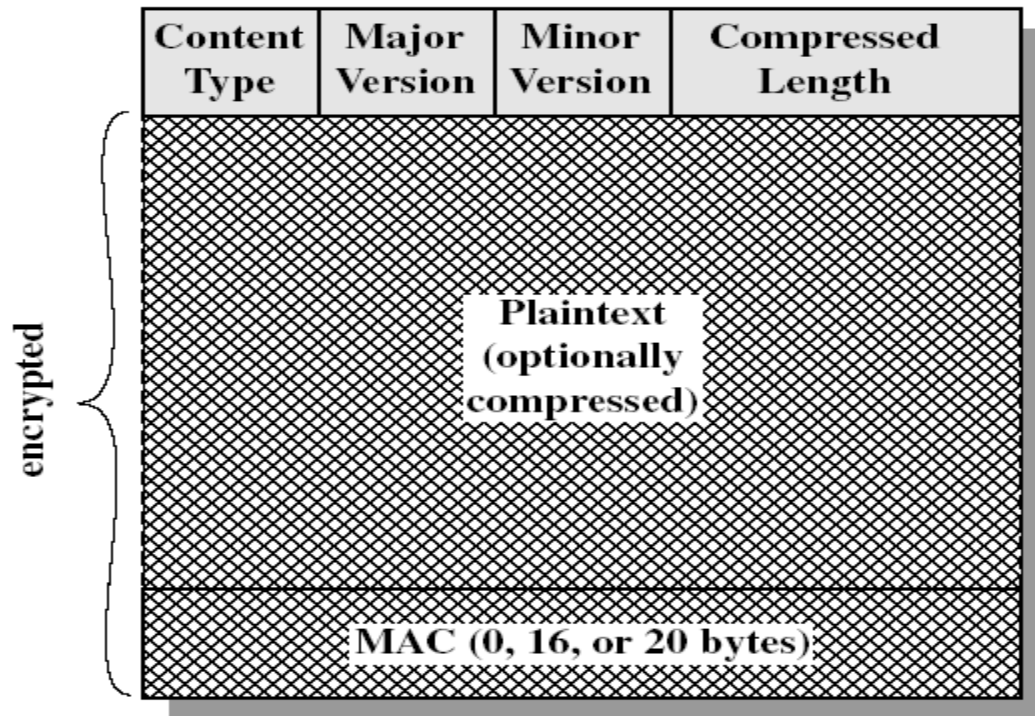


Figure 7.4 SSL Record Format

The content types defined are

- 1. change_cipher_spec,*
- 2. alert,*
- 3. handshake, and*
- 4. application data*

1. SSL Change Cipher Spec Protocol

- ✓ This protocol consists of a single message (Fig 7.5a), → a **single byte** with **value 1**.
- ✓ Purpose of this message: Copy the pending state into the current state, which updates the cipher suite to be used on this connection.
- ✓ This signal is used as a **coordination signal**.
- ✓ The client must send it to the server and the server must send it to the client.
- ✓ After each side has received it, all of the following messages are sent using the agreed-upon ciphers and keys.

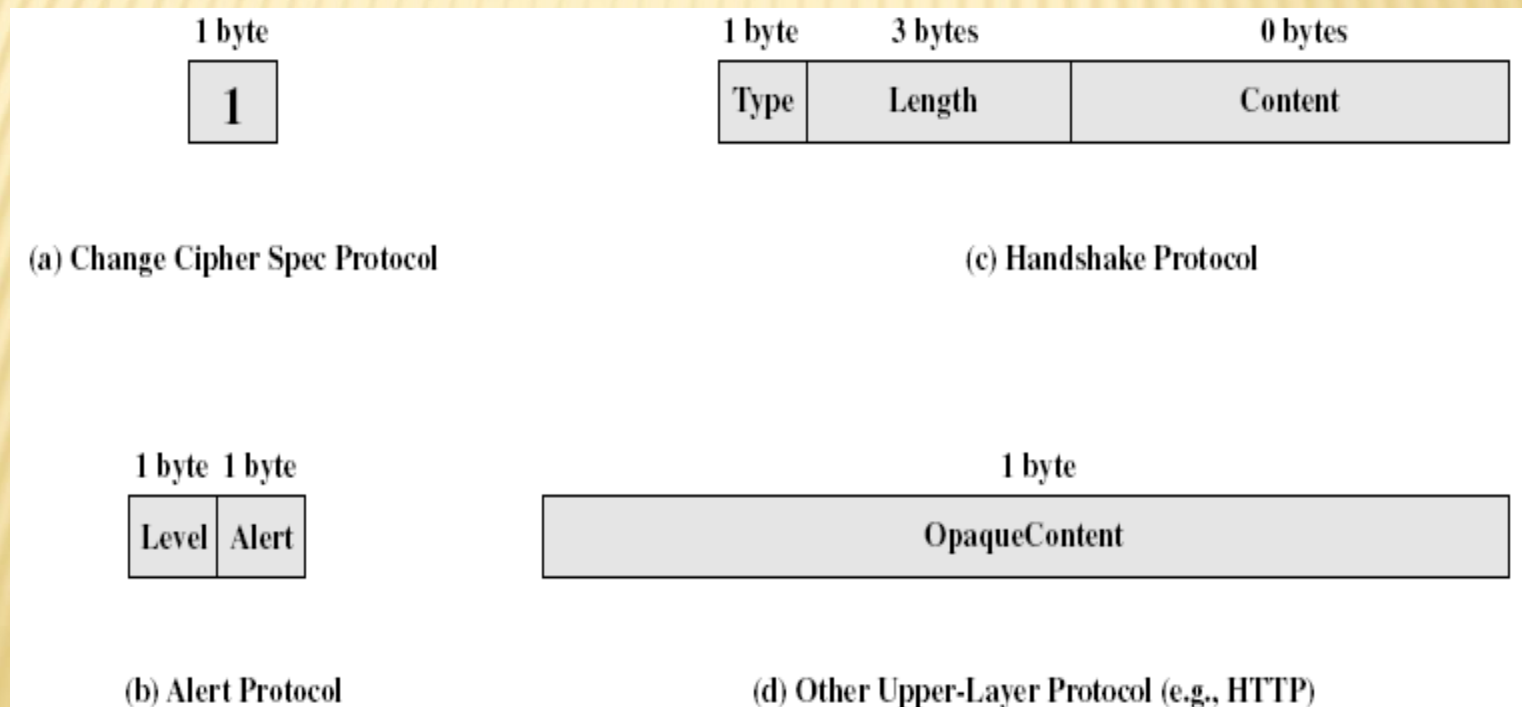
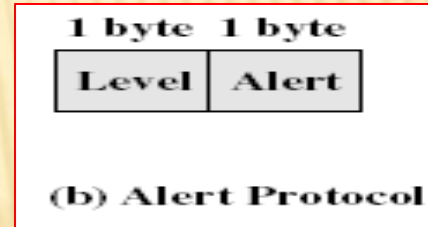


Figure 7.5 SSL Record Protocol Payload

2. Alert Protocol

- ✓ *Purpose: To convey SSL-related alerts to the peer entity. Alert messages are compressed and encrypted, as specified by the current state.*
- ✓ *Each message in this protocol consists of two bytes(Fig 7.5b).*
- ✓ *The first byte takes the value **warning(1)** or **fatal(2)** to convey the severity of the message. If **fatal**, SSL immediately **terminates the connection**. Other connections on the same session may continue, but no new connections on this session may be established.*
- ✓ *The second byte contains a code that indicates the specific alert.*



<i>Fatal Alerts are</i>	<i>The remaining alerts are</i>
<i>unexpected_message</i>	<i>close_notify</i>
<i>bad_record_mac</i>	<i>no_certificate</i>
<i>decompression_failure</i>	<i>bad_certificate</i>
<i>handshake_failure</i>	<i>unsupported_certificate</i>
<i>illegal_parameter</i>	<i>certificate_revoked</i>
	<i>certificate_expired</i>
	<i>certificate_unknown</i>

3. Handshake Protocol

✓ *Purpose: Allows the server and client to **authenticate** each other and to **negotiate** an **encryption and MAC algorithm and cryptographic keys** to be used to protect data sent in an SSL record*

✓ *The handshake Protocol is used before any application data are transmitted*

✓ *The Handshake Protocol consists of a series of messages exchanged by client and server. All of these have the format shown in Fig 7.5c. Each message has three fields*

Type(1 byte): Indicates one of 10 messages (table 7.2)

Length(3 bytes): The length of the message in bytes

Content(≥ 1 byte): The parameters associated with this message (table 7.2)

1 byte	3 bytes	≥ 0 bytes
Type	Length	Content

(c) Handshake Protocol

Table 7.2 SSL Handshake Protocol Message Types

Message Type	Parameters
hello_request	null
client_hello	version, random, session id, cipher suite, compression method
server_hello	version, random, session id, cipher suite, compression method
certificate	chain of X.509v3 certificates
server_key_exchange	parameters, signature
certificate_request	type, authorities
server_done	null
certificate_verify	signature
client_key_exchange	parameters, signature
finished	hash value

✓ Initial exchange needed to establish a logical connection between client and server

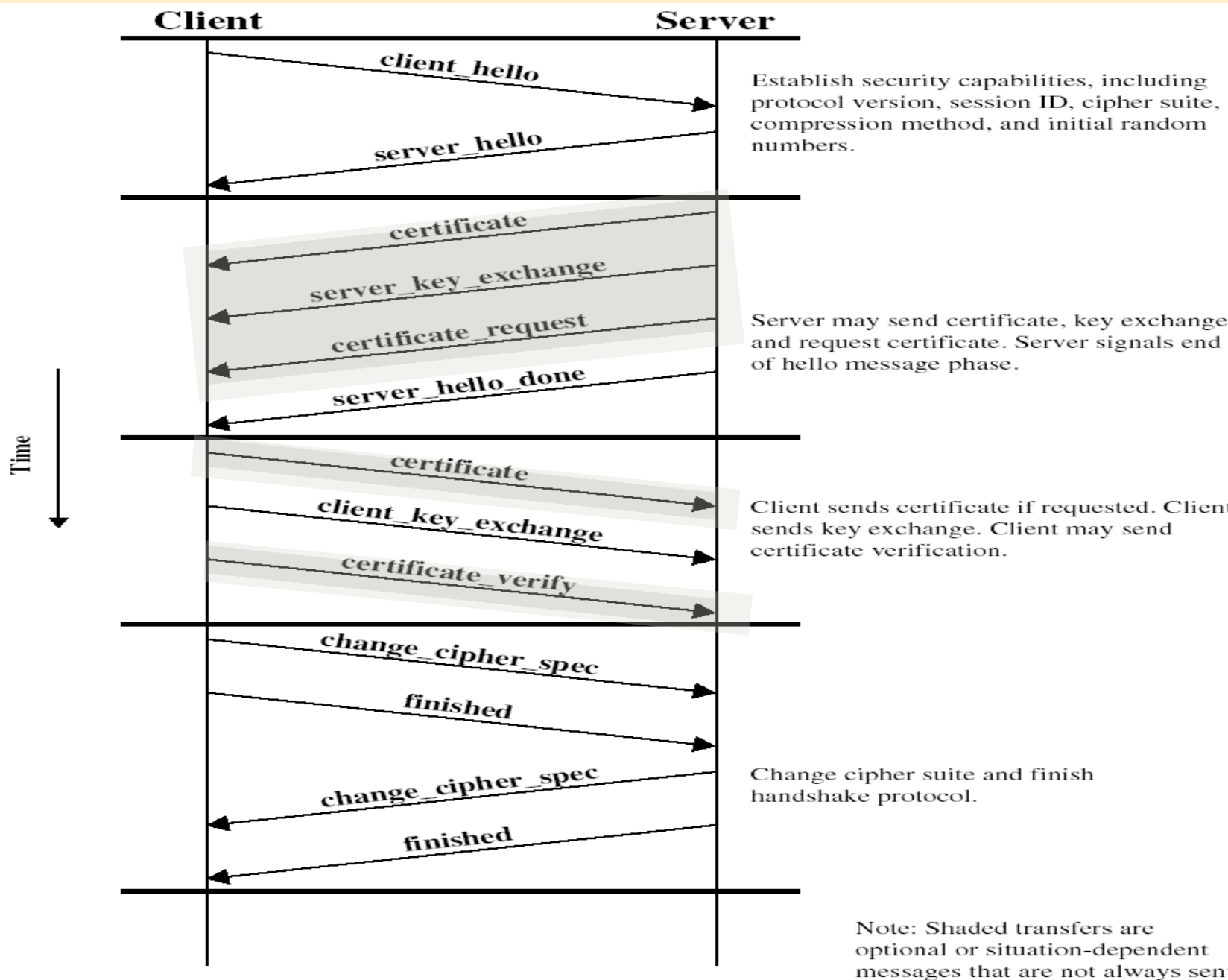


Figure 7.6 Handshake Protocol Action

✓ *This exchange has four phases*

Phase 1: Establish Security Capabilities

✓ *The exchange is initiated by the client, which sends a client_hello message with the following parameters*

Version, Random, Session ID, CipherSuite, Compression Method

✓ *After sending the client_hello message, the client waits for the server_hello message, which contains the same parameters as the client_hello message*

✓ *The 1st element of the CipherSuite parameter is the key exchange method (i.e., the means by which the cryptographic keys for conventional encryption and MAC are exchanged)*

RSA, Fixed Diffie-Hellman, Ephemeral Diffie-Hellman, Anonymous Diffie-Hellman, Fortezza

✓ *Following it is CipherSpec, which includes the fields:*

CipherAlgorithm, MAC algorithm, CipherType, IsExportable, HashSize, Key Material, IV Size

Phase 2: Server Authentication and Key Exchange

Phase 3: Client Authentication and Key Exchange

Phase 4: Finish

3. Transport Layer Security

✓ *TLS is an IETF standardization initiative whose goal is to produce an Internet standard version of SSL*

✓ *TLS is very similar to SSLv3*

Version Number: The TLS Record Format is the same as that of the SSL Record Format

Message Authentication Code: There are two differences between the SSLv3 and TLS MAC schemes:

- 1. Actual algorithm and*
- 2. Scope of the MAC calculation.*

✓ *TLS makes use of HMAC algorithm*

$$\text{HMAC}_k = H[(K^+ \oplus \text{opad}) \parallel H[(K^+ \oplus \text{ipad}) \parallel M]] \text{ (original of Unit -2)}$$

where

H = embedded hash function(MD5 or SHA-1)

M = message input to HMAC

K⁺ = secret key padded with zeros on the left(for MD5 and SHA-1, block length=512 bits)

ipad = 00110110(0x36) repeated 64 times(512 bits)

opad = 01011100(0x5C) repeated 64 times(512 bits)

Note: *SSLv3 uses the same algorithm, except that the padding bytes are concatenated with the secret key rather than being XORed with the secret key*

✓ *For TLS, the MAC is calculated over the following fields*

*HMAC_hash(MAC_write_secret, seq_num || TLSCompressed.type ||
TLSCompressed.version || TLSCompressed.length || TLSCompressed.fragment)*

✓ *The MAC calculation covers all of the fields covered by the SSLv3 calculation, plus the field TLSCompressed.version*

Pseudorandom Function (PRF):

*TLS makes use of a pseudorandom function referred to as PRF to **expand secrets into blocks of data** for purposes of key generation or validation*

$$\begin{aligned} P_hash(secret, seed) = & HMAC_hash(secret, (A(1)||seed)) \parallel \\ & HMAC_hash(secret, (A(2)||seed)) \parallel \\ & HMAC_hash(secret, (A(3)||seed)) \parallel \dots \end{aligned}$$

where $A()$ is defined as

$$A(0) = seed$$

$$A(i) = HMAC_hash(secret, A(i-1))$$

✓ *This function makes use of the HMAC algorithm, with either MD5 or SHA-1*

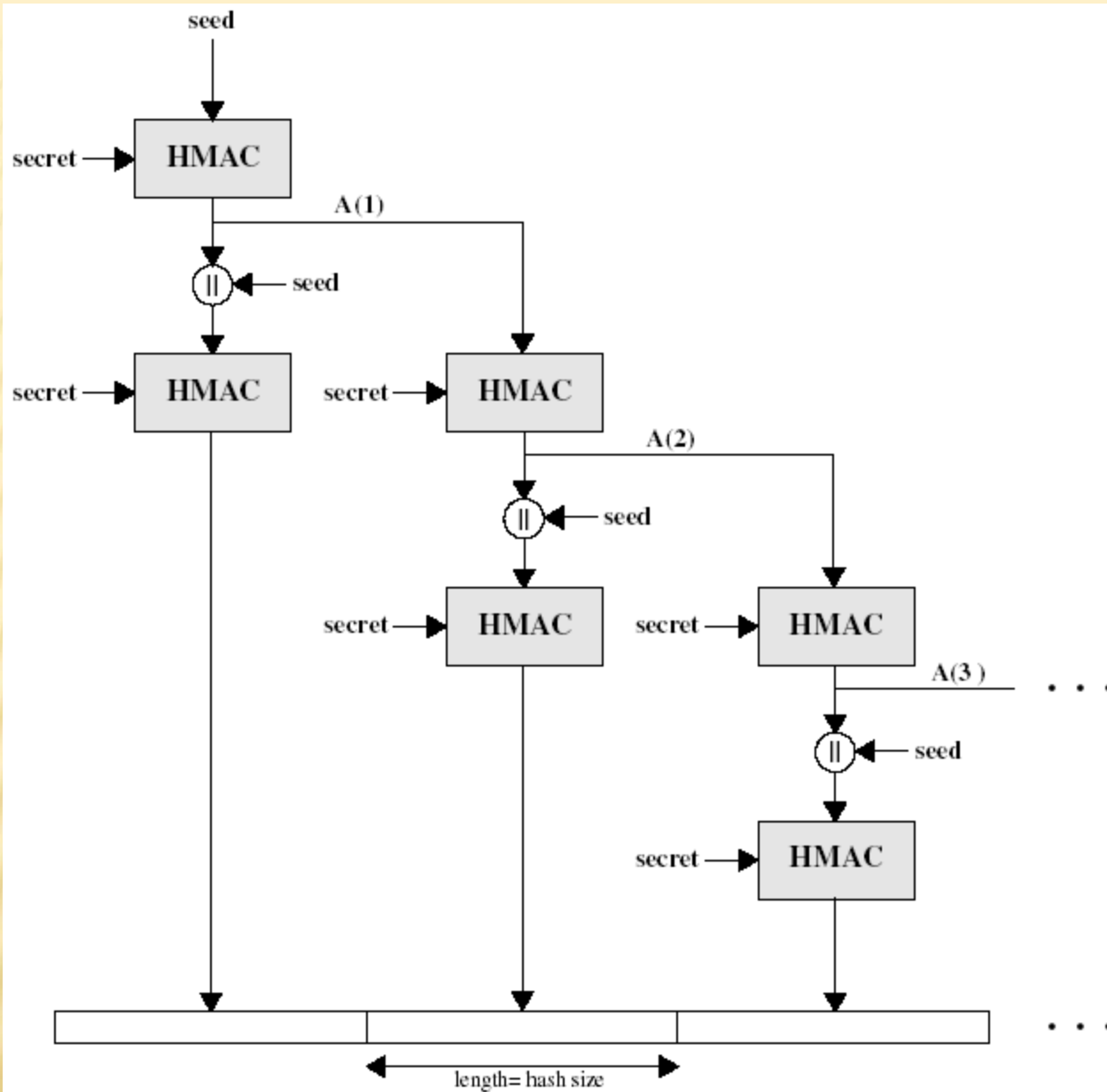


Figure 7.7 TLS Function P_hash (secret, seed)

✓ As can be seen, P_hash can be iterated as many times as necessary to produce the required quantity of data

✓ Each iteration involves *two executions of HMAC*, each of which in turn involves two executions of the underlying hash algorithm

✓ PRF is defined as

$PRF(secret, label, seed) =$

$P_MD5(S1, label || seed) \oplus P_SHA-1(S2, label || seed)$

Alert Codes: TLS supports all of the alert codes defined in SSLv3 with the exception of *no_certificate*. A number of additional codes are defined in TLS.

Additional Fatal alerts are:

- *decryption_failed*,
- *export_restriction*
- *record_overflow*,
- *protocol_version*,
- *unknown_ca*,
- *insufficient_security*,
- *access_denied*,
- *internal_error*
- *decode_error*,

Additional Warning alerts are

- *decrypt_error*,
- *user_canceled*,
- *no_renegotiation*

Cipher Suites in TLS:

There are several small differences between the cipher suites available under SSLv3 and under TLS:

Key Exchange: TLS supports all of the key exchange techniques of SSLv3 with the exception of Fortezza

Symmetric encryption algorithms: TLS includes all of the symmetric encryption algorithms found in SSLv3, with the exception of Fortezza

4. Secure Electronic Transaction (SET)

Definition

SET is an open encryption and security specification designed to protect credit card transactions on the Internet

Who initiated it?

Companies involved in developing the initial specification were IBM, Microsoft, Netscape, RSA, Terisa, and Verisign

What is it?

- *SET is **not a** payment system,*
- *It is a **set of security protocols and formats** that enables users to employ the existing credit card payment infrastructure on an open network, such as the Internet, in a secure fashion*

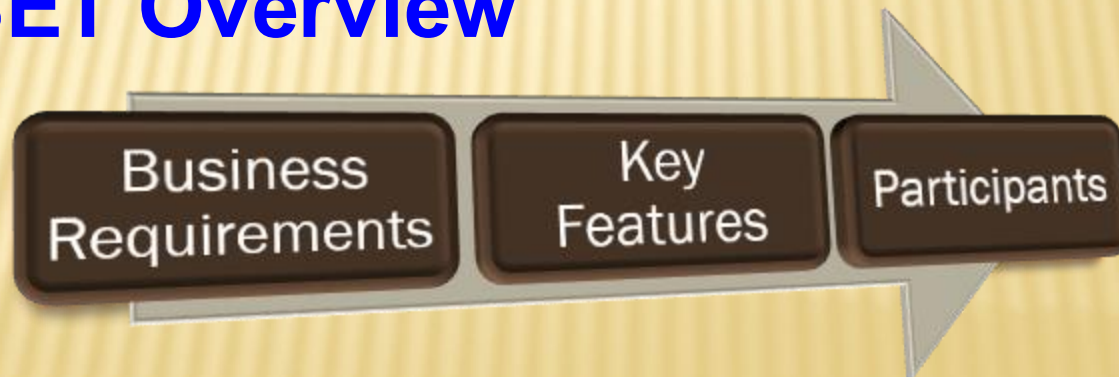
3 services of SET

*Provides a **secure communication** channel among all parties involved in a transaction*

*Provides **trust** by the use of X.509v3 digital certificates*

*Ensure **privacy** because the information is only available to parties in a transaction when and where necessary*

SET Overview



SET Overview: Requirements

1. ***Provide confidentiality of payment and ordering information***
2. ***Ensure the integrity of all transmitted data***
3. ***Provide authentication that a cardholder is a legitimate user of a credit card account***
4. ***Provide authentication that a merchant can accept credit card transactions through its relationship with a financial institution***
5. ***Ensure the use of the best security practices + system design techs to protect all legitimate parties in an e-commerce transaction***
6. ***Create a protocol that neither depends on transport security mechanisms nor prevents their use.***
7. ***Facilitate and encourage interoperability among software and network providers***

SET Overview: Key Features

To meet the requirements, SET incorporates the following features

*Confidentiality of
information*

DES is used

Integrity of data

*RSA digital signatures
+ SHA-1 hash code*

*RSA digital signatures
+ HMAC using SHA-1*

*Cardholder
account
authentication*

*X.509v3
digital
certificates +
RSA
signatures*

*Merchant
authentication*

*X.509v3
digital
certificates +
RSA
signatures*

SET Overview: Participants

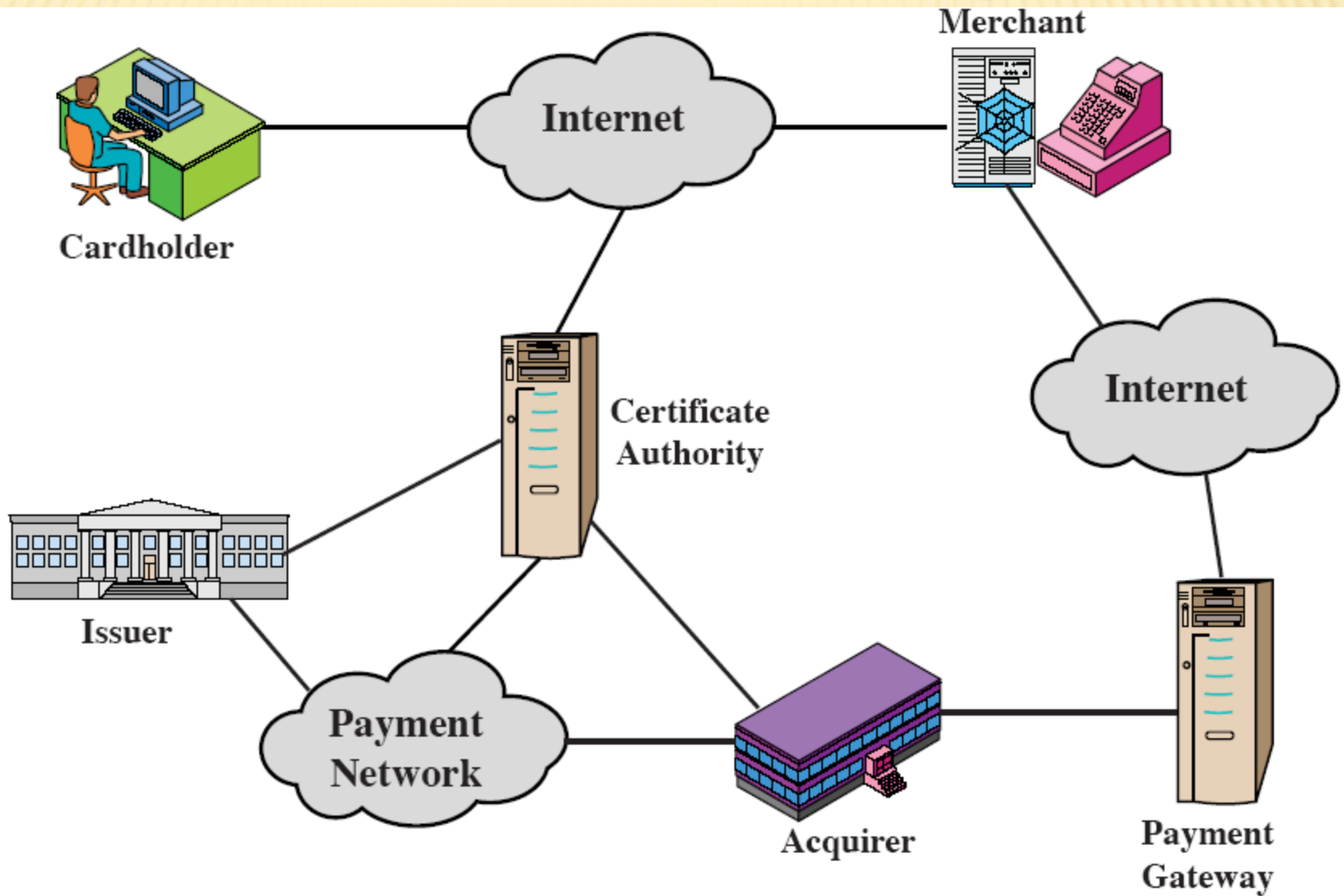


Figure 17.8 Secure Electronic Commerce Components

SET Participants

Cardholder: *A cardholder is an authorized holder of a payment card (e.g., MasterCard, Visa)*

Merchant: *A merchant is a person or organization that has goods or services to sell to the cardholder*

Issuer: *This is a financial institution, such as a bank, that provides the cardholder with the payment card*

Acquirer: *This is a financial institution that establishes an account with a merchant and processes payment card authorizations and payments*

Payment Gateway: *This is a function operated by the acquirer or a third party that processes merchant payment messages*

Certification Authority(CA): *This is an entity that is trusted to issue X.509v3 public-key certificates for cardholders, merchants, and payment gateways*

Sequence of events for a transaction

- 1. Customer opens an account*
- 2. Customer receives a certificate*
 - X.509v3 digital certificate, which is signed by the bank*
- 3. Merchants have their own certificates*
 - have two certificates, one for signing messages, and one for key exchange*
- 4. Customer places an order*
- 5. Merchant is verified*
- 6. Order and payment is sent*
- 7. Merchant requests payment authorization*
- 8. Customer confirms the order and Merchant acknowledges*
- 9. Merchant provides the goods or service*
- 10. Merchant requests payment*

Dual Signature

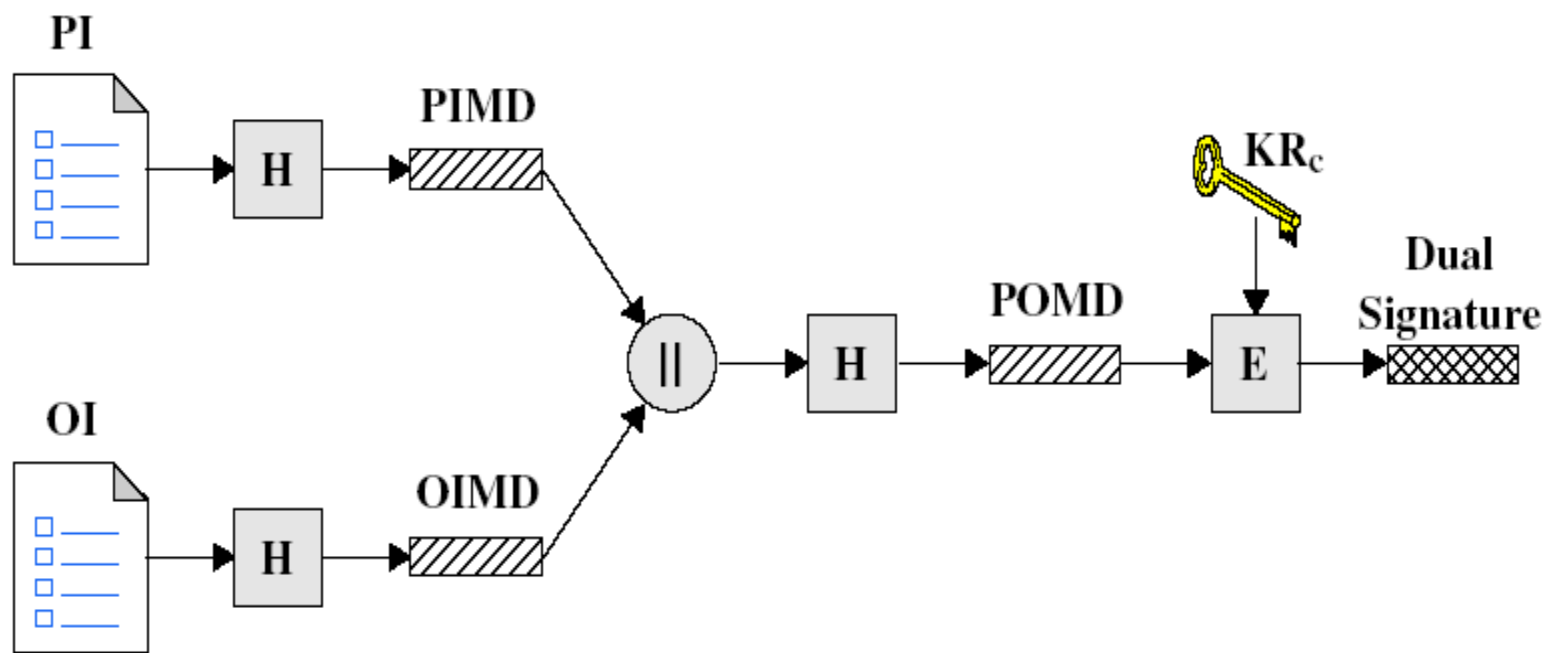
Purpose : *Link two messages that are intended for two different recipients*

✓ *The customer want to send the order information(OI) to the merchant and the payment information(PI) to the bank*

✓ *Customers send two messages to the merchant: a signed OI and a signed PI, and the merchant passes the PI on to the bank.*

Motivation: *The merchant may change the OI1 to OI2 and link the PI1 to OI2.*

✓ *They need to be linked to address the disputed later, if any.*



PI = Payment Information	PIMD = PI message digest
OI = Order Information	OIMD = OI message digest
H = Hash function (SHA-1)	POMD = Payment Order message digest
= Concatenation	E = Encryption (RSA)
	KR _c = Customer's private signature key

Figure 7.9 Construction of Dual Signature

✓ *The operation can be summarized as follows:*

$$DS = E_{KR_c} [H(H(PI) || H(OI))]]$$

Case 1: Consider if the merchant is in possession of DS, OI, and the PIMD.

✓ The merchant also has the public key of the customer, taken from the customer's certificate

✓ Then the merchant can compute the following quantities

$$H(PIMD \parallel H(OI)) \text{ and } D_{KU_c}[DS]$$

if these two quantities are equal, then the merchant has verified the signature

Case 2: If the bank is in possession of DS, PI, OIMD, and the customer's public key, then the bank can compute the following

$$H(H(PI) \parallel OIMD) \text{ and } D_{KU_c}[DS]$$

if these two quantities are equal, then the bank has verified the signature

SET supports different transaction types. They are

SET Supported Transactions

- card holder registration
- merchant registration
- purchase request
- payment authorization
- payment capture
- certificate query
- purchase inquiry
- purchase notification
- sale transaction
- authorization reversal
- capture reversal
- credit / payment reversal

Payment Processing

Purchase Request – Message from customer to merchant containing OI for merchant and PI for bank.

- ✓ *Before the purchase request exchange begins, the cardholder has completed browsing, selecting, and ordering*
- ✓ *The purchase request exchange consists of four messages: Initiate Request, Initiate Response, Purchase Request, and Purchase Response*
- ✓ *Initiate Request message: The customer requests the certificates (merchant and payment gateway) to the merchant.*
- ✓ *The Initiate Response message : Merchant responds with his certificates and payment gateway*
- ✓ *Next, the cardholder prepares the Purchase Request message (Fig 7.10)*

✓ *Purchase Request message includes the following:*

1.Payment-related information 2.Order-related information 3.Cardholder certificate

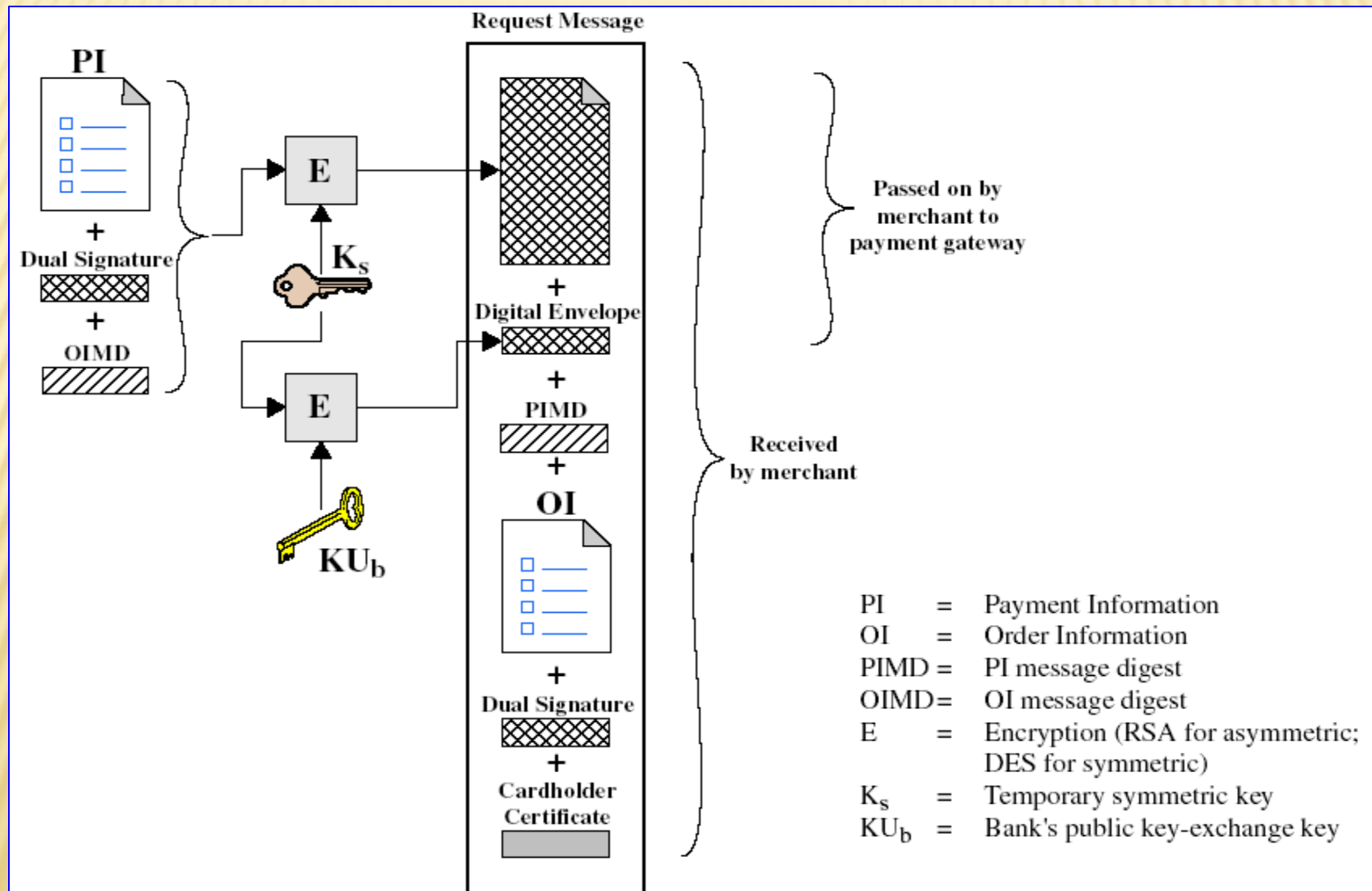


Figure 7.10 Cardholder Sends Purchase Request

✓ When the merchant receives the Purchase Request message, it performs the following

1. Verifies the cardholder
2. Forwards the payment information to the payment gateway for authorization
3. Sends a purchase response to the cardholder

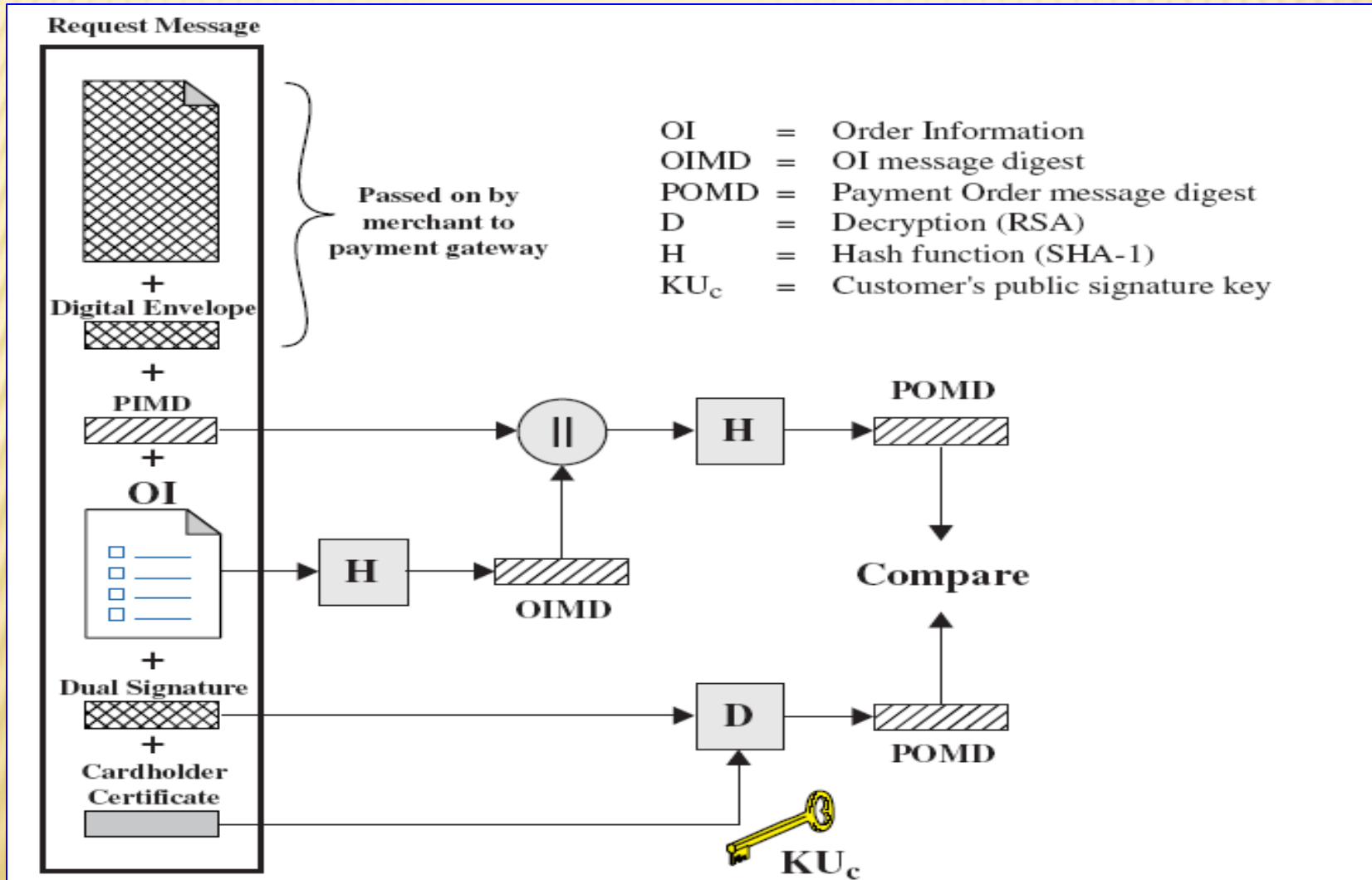


Figure 7.11 Merchant Verifies Customer Purchase Request

- ✓ *The Purchase Response message = $DS(KR_M, \text{response block that acknowledges the order})$*
- ✓ *The block and its signature are sent to the customer, along with the merchant's signature certificate*
- ✓ *When the cardholder receives the Purchase Response message, it verifies the signature on the response block*

Payment Authorization

Exchange between merchant and payment gateway to authorize a given amount for a purchase on a given Credit card a/c.

- ✓ *During the processing of an order from a cardholder, the merchant authorizes the transaction with the payment gateway. This authorization **guarantees that the merchant will receive payment**. The merchant can therefore provide the services or goods to the customer.*
- ✓ *The payment authorization exchange consists of two messages:
Authorization Request and Authorization Response*

✓ *The merchant sends an Authorization Request message to the payment gateway consisting of*

1. Purchase-related information = PI || DS || OIMD || Digital Envelope

2. Authorization-related information

- Authorization block includes transaction ID

- A digital envelope

3. Certificates

- Cardholder's certificate

- Merchant's signature key certificate

- Merchant's key exchange certificate (needed in the payment gateway's response)

✓ *The payment gateway performs the following tasks:*

1. Verifies all certificates

2. Decrypts the digital envelope and decrypts the authorization block

3. Verifies the Dual signature

✓ *The payment gateway returns an Authorization Response message to the merchant includes the following:*

1. Authorization-related information

- Authorization block

- Digital envelope

2. Capture token information

3. Certificate

- The gateway's signature key certificate

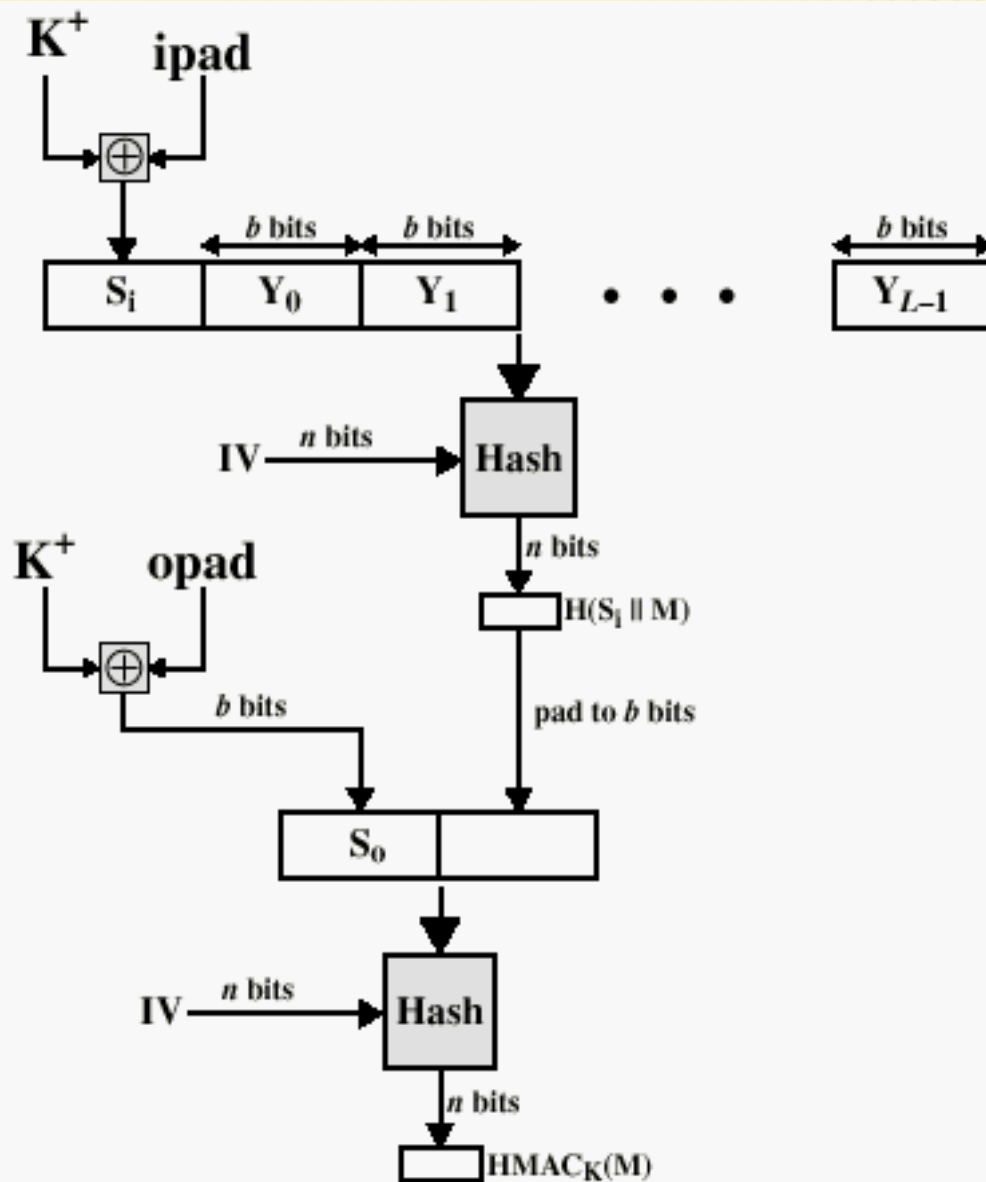
Payment Capture

- ✓ *To obtain payment*, the merchant engages the payment gateway in a payment capture transaction, consisting of a *capture request* and a *capture response message*
- ✓ *The Capture Request message includes the following:*
 - Capture request block = payment amount || transaction ID ||
Encrypted capture token || Merchant's certificates*
- ✓ *When the payment gateway receives the capture request message, it *decrypts and verifies* the capture request block and verifies the capture token block*
- ✓ *It then sends clearing request to the *issuer over the private network*. This request causes funds to be transferred to the merchant's account*
- ✓ *The gateway then notifies the merchant of payment in a Capture Response message. The message include the following:*
 - capture response block*
 - gateway's signature key certificate*

Appendix: HMAC (Unit II)

- **Use a MAC derived from a cryptographic hash code, such as SHA-1.**
- **Motivations:**
 - **Cryptographic hash functions executes faster in software than encryptoin algorithms such as DES**
 - **Library code for cryptographic hash functions is widely available**
 - **No export restrictions from the US.**
- **A cryptographic hash function: hash function which takes an input (or 'message') and returns a fixed-size alphanumeric string, which is called the hash value (sometimes called a message digest, a digital fingerprint, a digest or a checksum).**

HMAC Structure



- H = embedded hash function (e.g., SHA-1)
- M = message input to HMAC (including the padding specified in the embedded hash function)
- Y_i = i th block of M , $0 \leq i \leq (L-1)$
- L = number of blocks in M
- b = number of bits in a block
- n = length of hash code produced by embedded hash function
- K = secret key;
- K^+ = K padded with zeros on the left so that the result is b bits in length
- ipad = 00110110 (36 in hexadecimal) repeated $b/8$ times
- opad = 01011100 (5C in hexadecimal) repeated $b/8$ times
- Then HMAC can be expressed as follows:
- $$\text{HMAC}_k = H[(K^+ \oplus \text{opad}) || H[(K^+ \oplus \text{ipad}) || M]]$$

- HMAC Algorithm

1. Append zeros to the left end of K to create a b -bit string K^+ (e.g., if K is of length 160 bits and $b=512$, then K will be appended with 44 zero bytes $0x00$).
2. XOR K^+ with $ipad$ to produce the b -bit block s_i
3. Append M to s_i
4. Apply H to the stream generated in step3
5. XOR K^+ with $opad$ to produce the b -bit block s_0
6. Append the hash result from step4 to s_0
7. Apply H to the stream generated in step6 and output the result.